

MÃO NA MASSA:

O Guardião do Cadastro da Black Friday

Contexto:

Nossa loja virtual está se preparando para a Black Friday! Para que as ofertas cheguem aos clientes reais, precisamos garantir que o formulário de cadastro não aceite "dados lixo" (como e-mails falsos ou senhas fracas). O seu objetivo é construir um protótipo de formulário que faça essa validação.

O que vamos aprender?

Como capturar dados de um formulário, evitar o recarregamento da página e usar uma RegEx para validar um e-mail.

Passo 1: A Estrutura Visual (HTML)

Primeiro, vamos criar a "cara" do nosso formulário. Crie um arquivo `index.html`.

Observe que criamos um texto de erro (o `span`), mas ele começa escondido (`display:none`). Nós só vamos mostrá-lo se o usuário errar!

Aqui está a estrutura HTML completa e pronta para uso:

HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Cadastro - Black Friday</title>
  <style>
    /* Um pouco de CSS básico apenas para deixar o formulário mais organizado visualmente */
    body {
      font-family: Arial, sans-serif;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      background-color: #f4f4f9;
    }
    form {
      background: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      display: flex;
      flex-direction: column;
      gap: 15px;
      width: 300px;
    }
    input {
      padding: 10px;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    button {
```

```

padding: 10px;
background-color: #28a745;
color: white;
border: none;
border-radius: 4px;
cursor: pointer;
}
button:hover {
  background-color: #218838;
}
</style>
</head>
<body>

<form id="formCadastro">
  <h3>Cadastro Black Friday</h3>

  <input type="email" id="email" placeholder="Seu e-mail">
  <span id="erroEmail" style="color:red; display:none; font-size: 14px;">E-mail inválido!</span>

  <input type="password" id="senha" placeholder="Sua senha (mín. 6 caracteres)">

  <button type="submit">Cadastrar</button>
</form>

<script src="script.js"></script>
</body>
</html>

```

Passo 2: Assumindo o Controle (JavaScript)

Por padrão, quando clicamos em "submit", o navegador recarrega a página. Nós não queremos isso! Crie seu arquivo `script.js` e use o comando `e.preventDefault()` para dizer ao navegador: *"Deixe que eu assumo o controle daqui"*.

JavaScript

```

const form = document.querySelector('#formCadastro');

// Fica "escutando" o momento em que o botão de submit for clicado
form.addEventListener('submit', (e) => {
  e.preventDefault(); // Impede a página de recarregar
  console.log("Iniciando validação...");

  // Os próximos passos vão entrar AQUI dentro!
});

```

Passo 3: O Teste do "Molde" (Regex)

Agora vamos pegar o que o usuário digitou e testar no nosso "molde" de e-mail. Adicione este código dentro da função do Passo 2.

Como a Regex funciona aqui? A variável `regexEmail` verifica se existe um texto, seguido de um "@", seguido de mais texto, um ponto "." e o final do domínio.

JavaScript

```
const email = document.querySelector('#email').value;
const senha = document.querySelector('#senha').value;

// Nosso "molde" para e-mails válidos
const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
let formValido = true;

// Se o e-mail NÃO passar no teste da RegEx...
if (!regexEmail.test(email)) {
  document.querySelector('#erroEmail').style.display = 'block'; // Mostra o erro
  formValido = false; // Reprova o formulário
} else {
  document.querySelector('#erroEmail').style.display = 'none'; // Esconde o erro
}
```

Passo 4: O Veredito Final

Ainda dentro da função, vamos conferir se o formulário passou no teste do e-mail E se a senha atende ao requisito mínimo de tamanho.

JavaScript

```
// Se o e-mail estiver válido E a senha tiver 6 ou mais caracteres...
if (formValido && senha.length >= 6) {
  alert("Cadastro realizado com sucesso! Dados prontos para o servidor.");
  form.reset(); // Limpa todos os campos para o próximo usuário
} else if (senha.length < 6) {
  alert("A senha precisa ter pelo menos 6 caracteres.");
}
```

Como deve ficar o seu JavaScript Completo?

Caso tenha se perdido nas etapas, seu arquivo `script.js` deve estar exatamente assim:

JavaScript

```
const form = document.querySelector('#formCadastro');

form.addEventListener('submit', (e) => {
  e.preventDefault();

  const email = document.querySelector('#email').value;
  const senha = document.querySelector('#senha').value;
  const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  let formValido = true;

  if (!regexEmail.test(email)) {
    document.querySelector('#erroEmail').style.display = 'block';
    formValido = false;
  } else {
    document.querySelector('#erroEmail').style.display = 'none';
  }

  if (formValido && senha.length >= 6) {
```

```
    alert("Cadastro realizado com sucesso! Dados prontos para o servidor.");
    form.reset();
} else if (senha.length < 6) {
    alert("A senha precisa ter pelo menos 6 caracteres.");
}
});
```

Resultado Esperado: Ao testar no navegador, se você digitar "https://www.google.com/search?q=joao.gmail.com" (sem @), a mensagem vermelha deve aparecer. Se preencher tudo certinho, um alerta de sucesso vai pular na tela e os campos ficarão em branco novamente.

Este exercício, intitulado "**O Guardião do Cadastro da Black Friday**", é uma excelente introdução prática ao desenvolvimento web front-end. Ele foca em como tornar uma página estática em uma aplicação interativa e funcional, garantindo que apenas dados válidos cheguem ao sistema.

Abaixo, apresento uma explicação didática dividida pelos pilares do projeto:

O Objetivo: Qualidade de Dados

Em eventos de alto tráfego como a **Black Friday**, as empresas não podem perder tempo ou recursos com cadastros falsos. O exercício ensina a criar uma "barreira" no lado do cliente (o navegador) para validar informações antes mesmo de enviá-las a um servidor.

A Estrutura Visual (O "Corpo")

O código HTML define os elementos que o usuário vê:

- **Campos de entrada (input):** Para coletar o e-mail e a senha.
- **Feedback visual imediato:** O uso de um `<span>` com `display: none` é uma técnica comum. Ele está lá o tempo todo, mas "invisível", esperando o comando do JavaScript para aparecer caso o usuário cometa um erro.

O Comportamento (A "Inteligência")

O arquivo `script.js` é onde a mágica acontece. Ele segue três etapas lógicas cruciais:

A. Interrupção do Comportamento Padrão

Por padrão, formulários recarregam a página ao serem enviados. O comando `e.preventDefault()` é a "chave" que trava esse comportamento, permitindo que o JavaScript valide os dados sem que a página "pisque" ou limpe os campos antes da hora.

B. O Molde Mágico: RegEx

A validação de e-mail utiliza uma **Expressão Regular (RegEx)**.

A variável `regexEmail` funciona como uma máscara de busca que verifica se o texto segue o padrão obrigatório: texto + @ + texto + . + final do domínio. Se o que foi digitado não se encaixar nesse molde, o erro é exibido.

C. Lógica de Condicional Dupla

O formulário só é considerado "aprovado" se passar em dois testes simultâneos:

1. **O e-mail deve ser válido** (conforme a RegEx).
2. **A senha deve ter tamanho mínimo** (pelo menos 6 caracteres).

O Fluxo de Trabalho do Código

O exercício demonstra de forma clara o fluxo de manipulação do DOM (Document Object Model):

- 1. **Captura:** O JS busca os valores digitados no HTML via `document.querySelector().value`.
- 2. **Processamento:** Aplica os testes lógicos e matemáticos (como `.length` para a senha).
- 3. **Resposta:** Altera o estilo do HTML (exibe o erro) ou limpa o formulário (`form.reset()`) em caso de sucesso.

Resumo da Atividade

Conceito	Aplicação Prática no Exercício
UX (Experiência do Usuário)	Mostrar mensagens de erro específicas sem recarregar a página.
Segurança Básica	Garantir que senhas não sejam curtas demais.
Limpeza de Estado	Resetar o formulário após o sucesso para um novo uso.

Este exercício é fundamental porque reflete um cenário real de desenvolvimento, onde o programador atua como o "guardião" que protege a integridade do banco de dados da aplicação. O que você achou da lógica de usar o `preventDefault()`?